

Lecture 3:

The Relational Data Model

Topic Objectives

- Learn the conceptual foundation of the relational model
- Understand *how relations* differ from non-relational tables
- Learn basic relational terminology
- Learn the meaning and the importance of keys, foreign keys, and related terminology
- Understand how foreign keys represent relationships
- Learn the purpose and use of surrogate keys
- Learn the meaning of functional dependencies
- Learn to apply a process for normalizing relations

Entity

- An entity is something of importance to a user or organization that needs to be represented in a database
- An entity represents one theme, topic, or business concept
- In the entity-relationship model, entities are restricted to things that can be represented using a single table

Relational Model Concepts

- A Relation is a mathematical concept based on the ideas of sets
- The model was first proposed by Dr. E.F. Codd of IBM Research in 1970 in the following paper:
 - *"A Relational Model for Large Shared Data Banks," Communications of the ACM, June 1970*
- The above paper caused a major revolution in the field of database management and earned Dr. Codd the coveted ACM Turing Award

Relation....

- A relation is a two-dimensional table that has specific characteristics
- The table dimensions, like a matrix, consist of rows and columns

Characteristics of a Relation

- Rows contain data about instances of an entity
- Columns contain data about attributes of the entity
- Cells of the table hold a single value
- All values in a column are of the same kind (data type)
- Each column has a unique name
- The order of the columns is unimportant
- The order of the rows is unimportant
- No two rows can be identical

Terminology

Synonyms...

Table	Row	Column
File (rarely)	Record	Field
Relation	Tuple (rarely)	Attribute

Informal Definitions

- Informally, a **relation** looks like a **table** of values.
- A relation typically contains a **set of rows**.
- The data elements in each **row** represent certain facts that correspond to a real-world **entity** or **relationship**
 - In the formal model, rows are called **tuples**
- Each **column** has a column header that gives an indication of the meaning of the data items in that column
 - In the formal model, the column header is called an **attribute name** (or just **attribute**)

- **Entity-Relationship (ER) model**
 - Popular high-level conceptual data model
- **ER diagrams**
 - Diagrammatic notation associated with the ER model
- **Unified Modeling Language (UML)**

Using High-Level Conceptual Data Models for Database Design

- **Requirements collection and analysis**
 - Database designers interview prospective database users to understand and document data requirements
 - Result: **data requirements**
 - **Functional requirements** of the application

A Sample Database Application

- COMPANY
 - Employees, departments, and projects
 - Company is organized into departments
 - Department controls a number of projects
 - Employee: store each employee's name, Social Security number, address, salary, sex (gender), and birth date
 - Keep track of the dependents of each employee

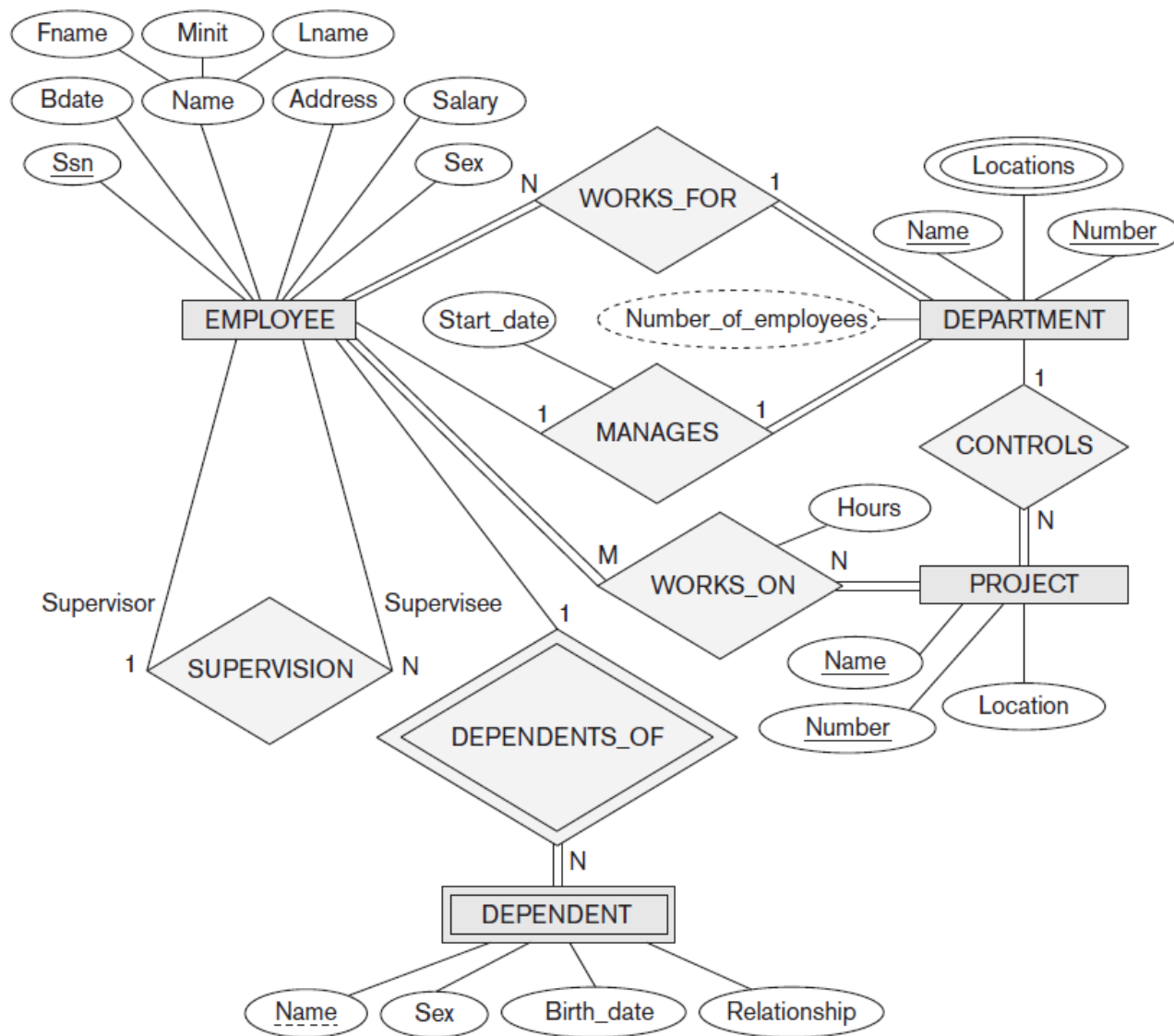


Figure 7.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 7.14.

Example of a Relation

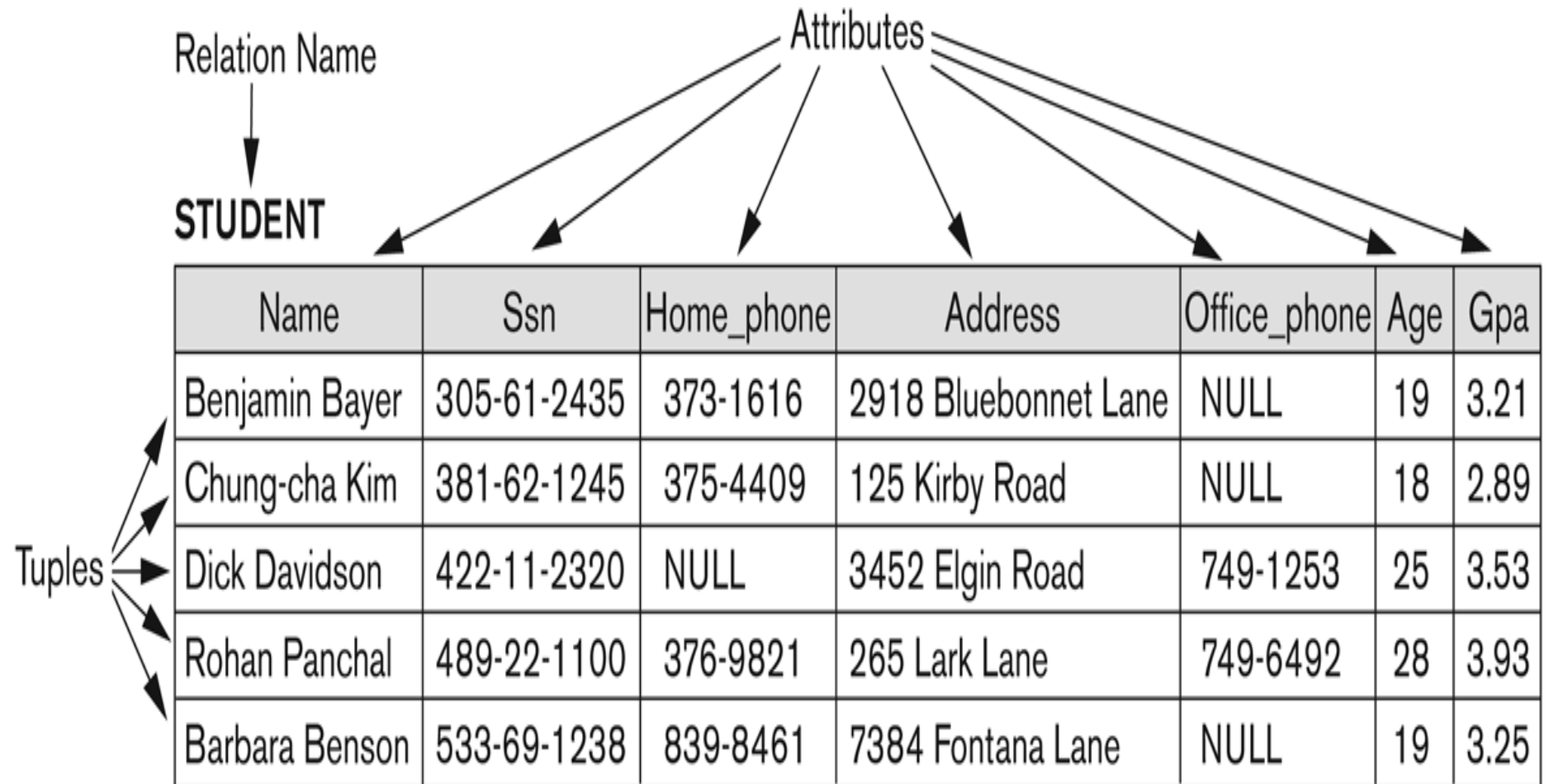


Figure 5.1

The attributes and tuples of a relation STUDENT.

Entity Types, Entity Sets, Attributes, and Keys

- ER model describes data as:
 - Entities
 - Relationships
 - Attributes

Entities and Attributes

- **Entity**
 - Thing in real world with independent existence
- **Attributes**
 - Particular properties that describe entity
 - Types of attributes:
 - *Composite versus simple (atomic) attributes*
 - **Single-valued** versus **multivalued** attributes
 - **Stored** versus **derived** attributes
 - **NULL** values
 - **Complex** attributes

Entities and Attributes (cont.)

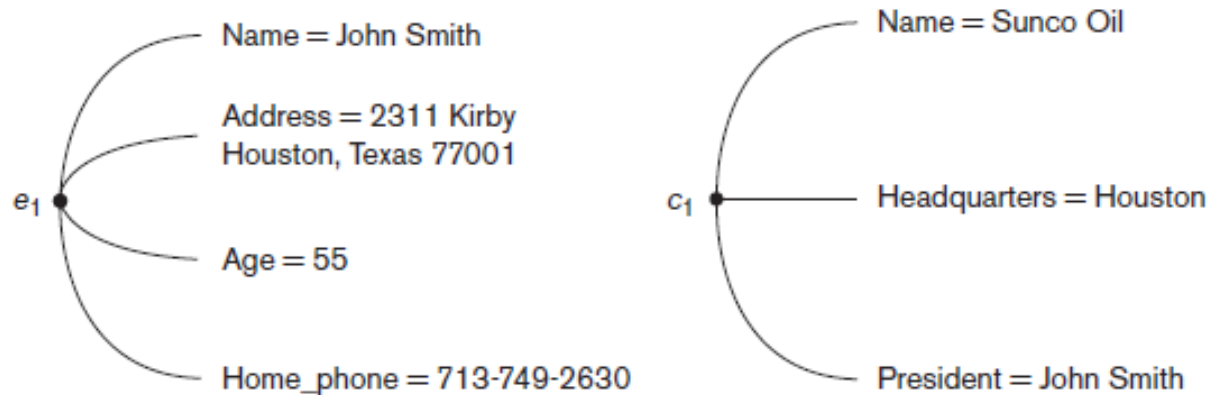


Figure 7.3
Two entities,
EMPLOYEE e_1 , and
COMPANY c_1 , and
their attributes.

Entity Types, Entity Sets, Keys, and Value Sets

- **Entity type**
 - Collection (or set) of entities that have the same attributes

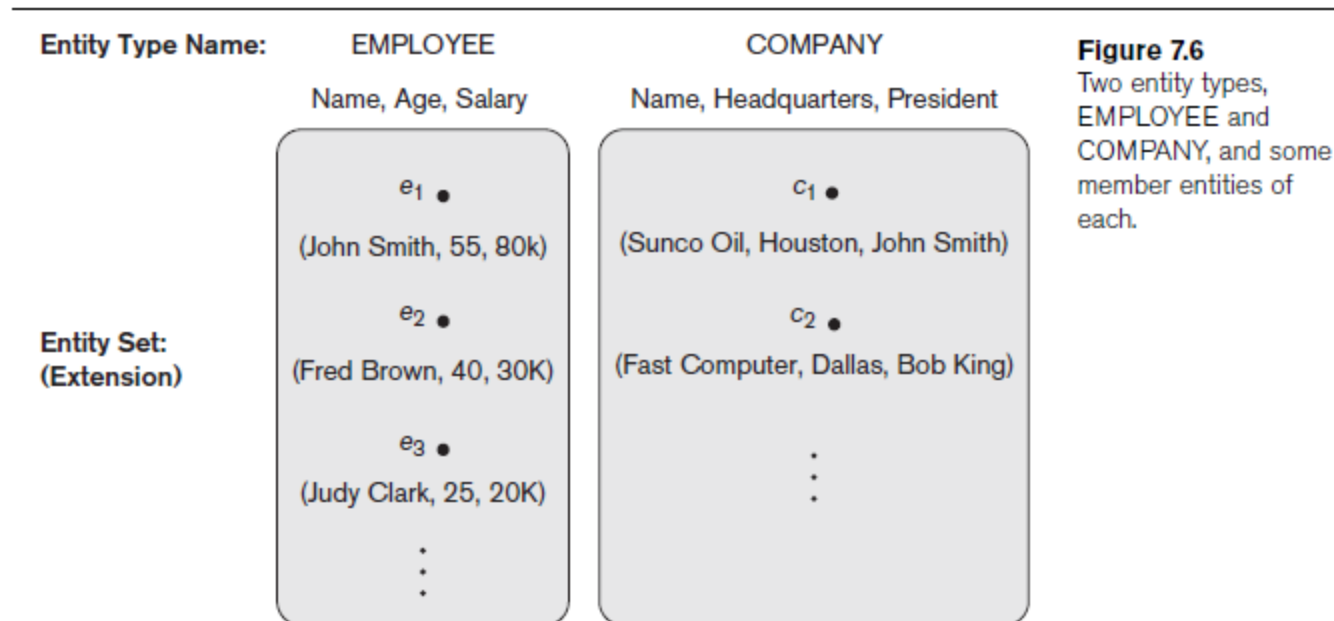


Figure 7.6
Two entity types, EMPLOYEE and COMPANY, and some member entities of each.

Entity Types, Entity Sets, Keys, and Value Sets (cont.)

- **Key or uniqueness constraint**
 - Attributes whose values are distinct for each individual entity in entity set
 - **Key attribute**
 - Uniqueness property must hold for every entity set of the entity type
- **Value sets (or domain of values)**
 - Specifies set of values that may be assigned to that attribute for each individual entity

Initial Conceptual Design of the COMPANY Database

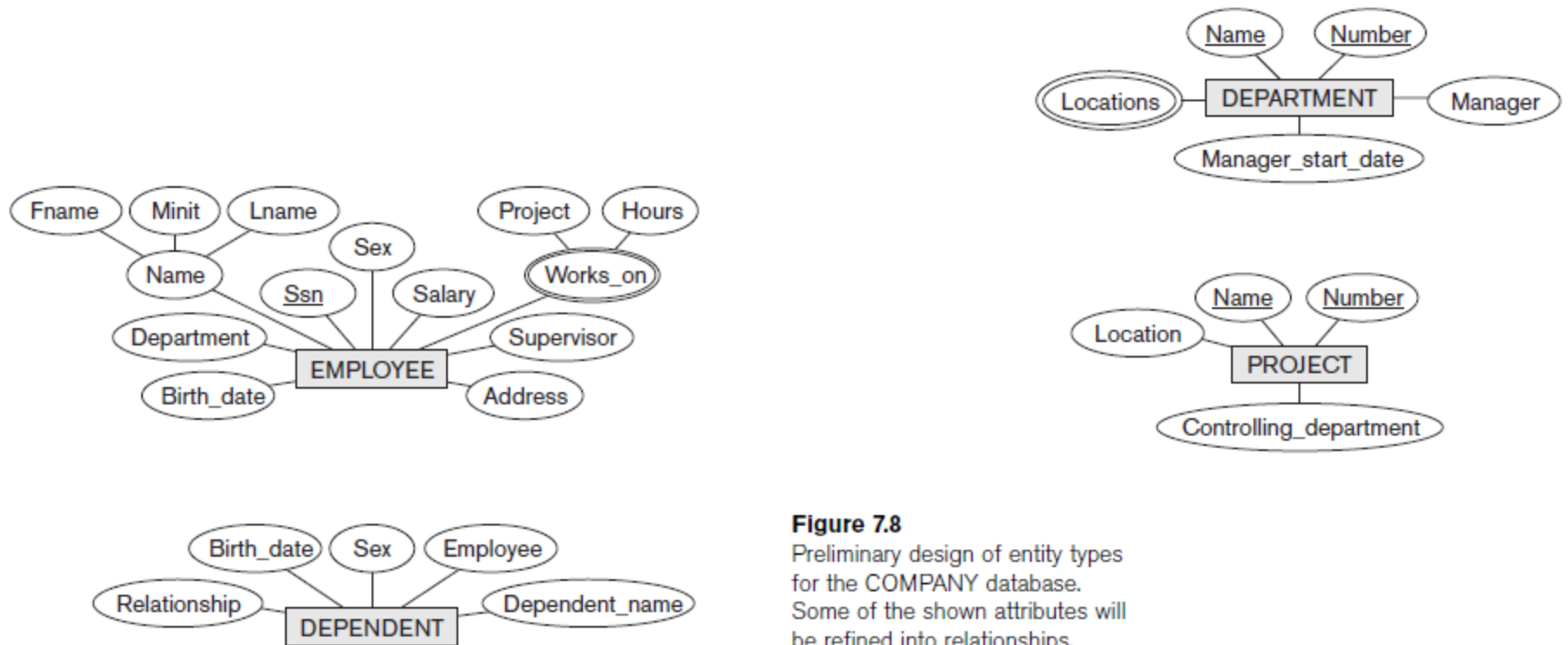


Figure 7.8

Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Informal Definitions

- Key of a Relation:
 - Each row has a value of a data item (or set of items) that uniquely identifies that row in the table
 - Called the *key*
 - In the STUDENT table, SSN is the key
 - Sometimes row-ids or sequential numbers are assigned as keys to identify the rows in a table
 - Called *artificial key* or *surrogate key*

Keys

- A Key is one (or more) columns of a relation whose values are used to identify a row

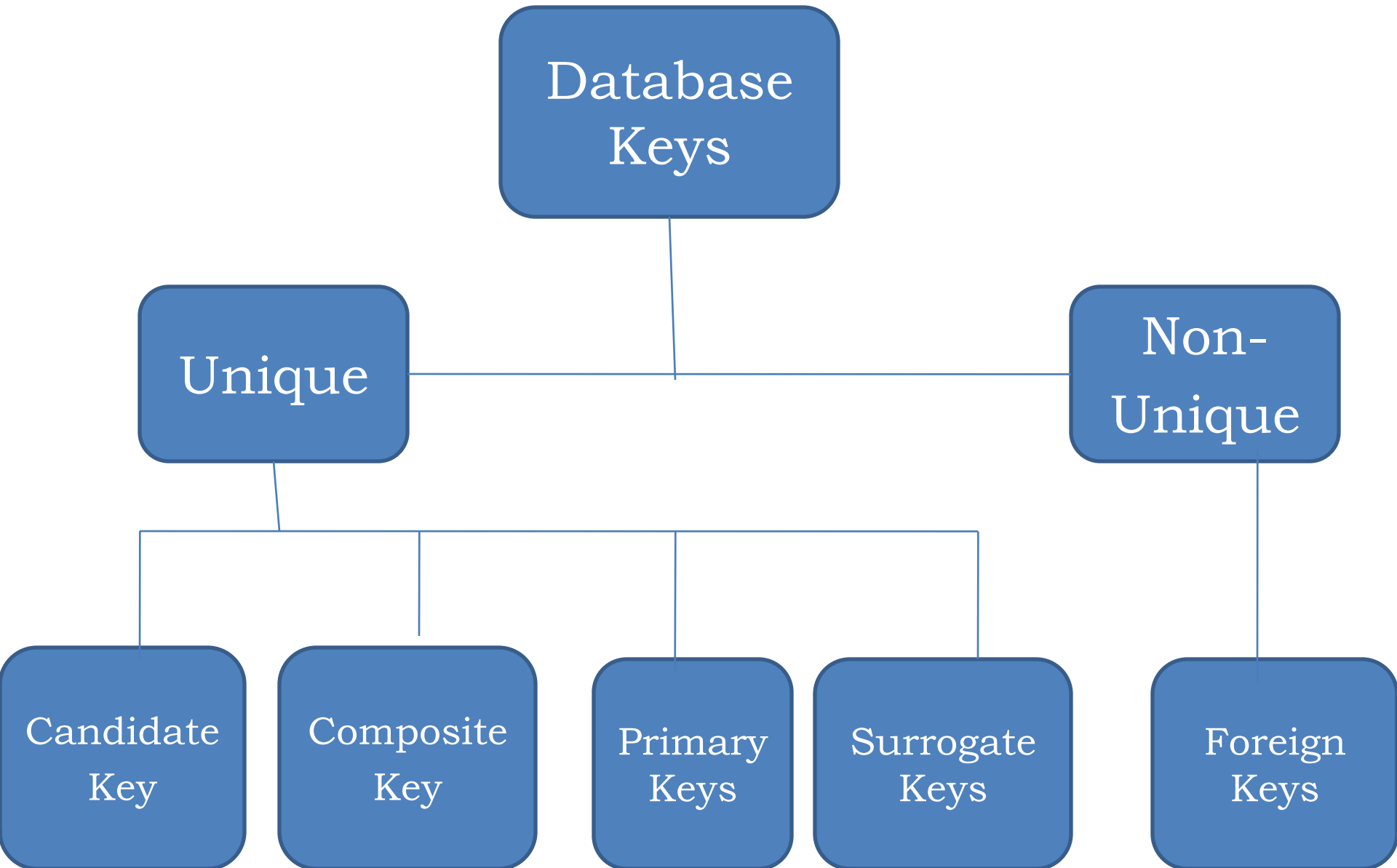
Uniqueness of Keys

Unique Keys	Non- Unique Keys
Data value is unique for each row	Data value may be shared among several rows.
Consequently, the key will uniquely identify a row	Consequently, the key will uniquely identify a row

Example of A non – key Unique

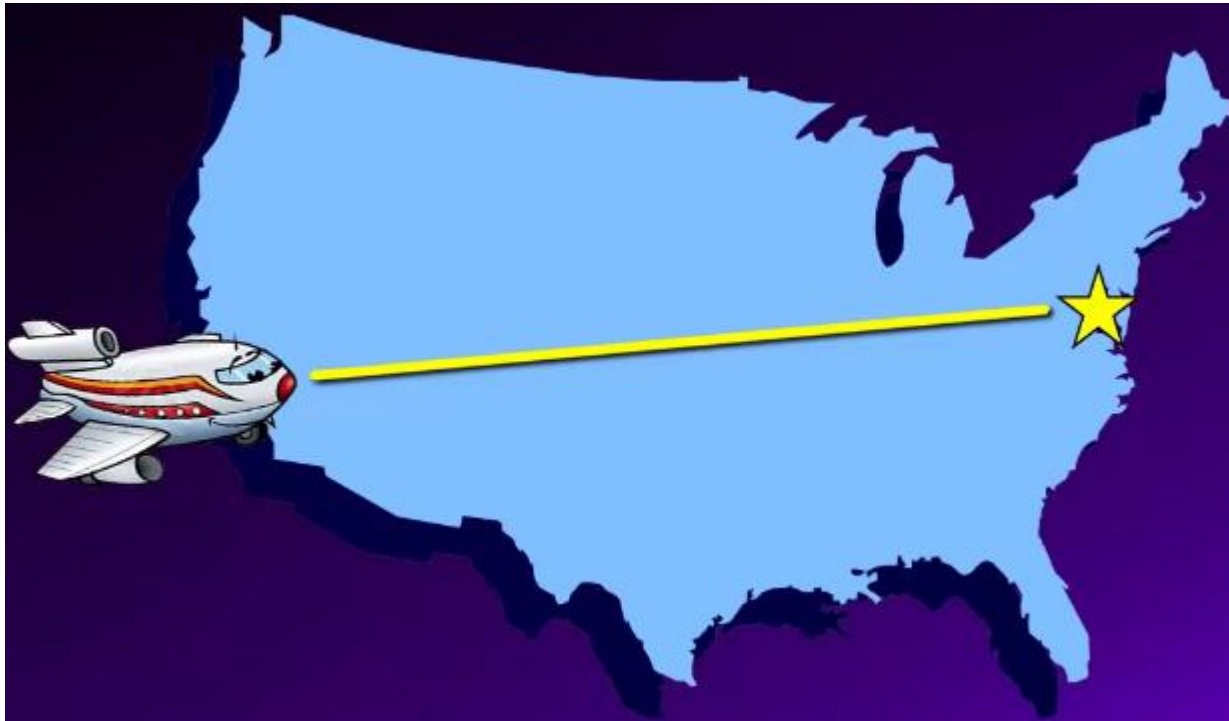
Employee ID	Name	Phone Number	Dept ID
101	Dani	714555-7273	1
102	Nyabo	6575557273	2
103	Nuru	787555-3247	2
104	Leah	754555-3247	2

Dan's Topology of Database Keys



A Composite Key

- A Composite Key is a key that composed of two or more attributes
- For a key to be unique, it must often become a composite key
- Composite Key Example



Composite Key Example



Flight Number	Date
UA 36	03 Sep 2021
AA 704	31 Oct 2021
UA 36	04 Sep 2021
BA 9	31 Oct 2021

A Candidate Key

- A Candidate Key is called “candidate” because it has the potential to become the primary key
- A Candidate Key is a unique key

A Primary Key

- A primary Key is a candidate key chosen to be the main key for the relation
- If you know the value of the primary key, you will be able to uniquely identify a single row within the table

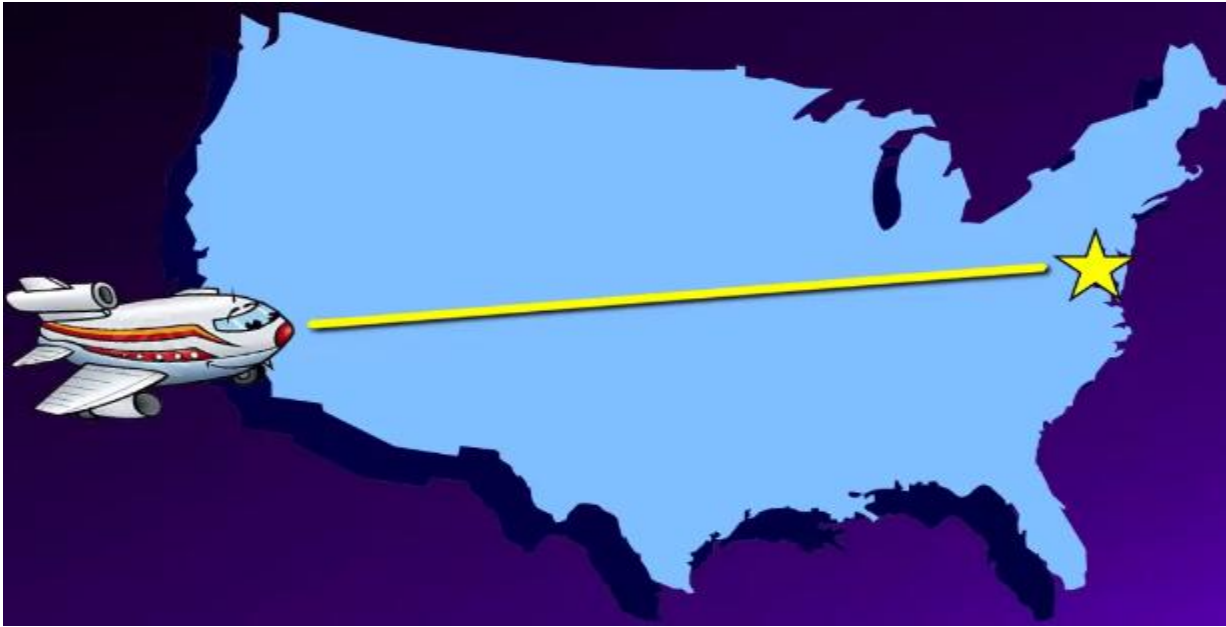
Primary Key Example

EmployeeNumber	FirstName	LastName
100	Aneth	Omega
101	James	Pius
102	Jerry	Candle
308	Daniel	Sinkonde

A Surrogate Key

- A surrogate key is a unique, numeric value that is added to a relation to serve as the primary key
- Surrogate key values have no meaning to users and are usually hidden on forms, queries, and reports
- A surrogate key is often used in place of a composite key

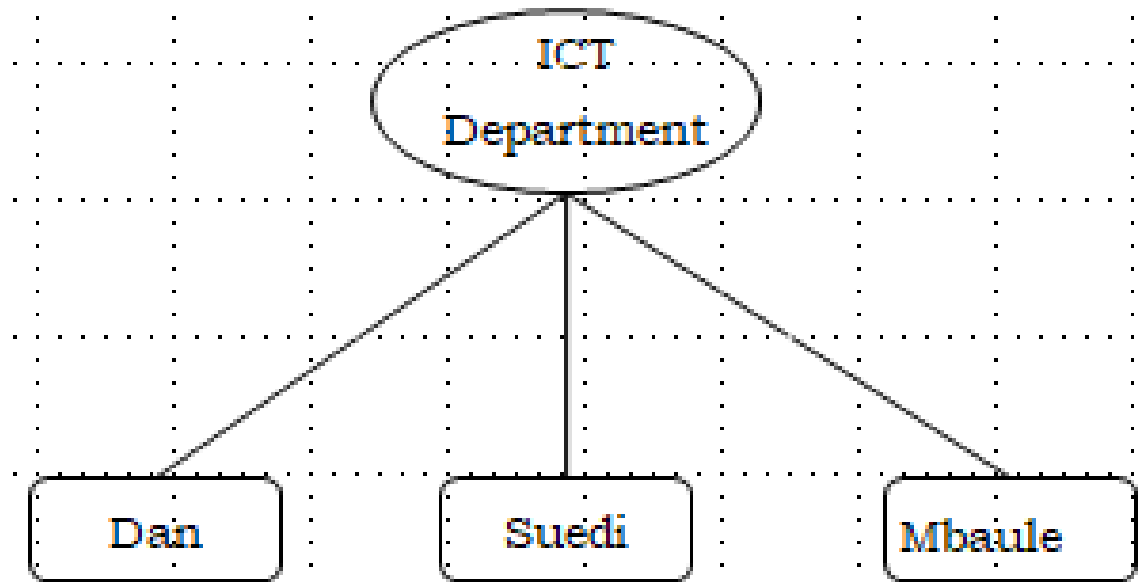
Surrogate Key Example



Flight ID	Flight Number	Date
1	UA 36	03 Sep 2021
2	AA 704	31 Oct 2021
3	UA 36	04 Sep 2021
4	BA 9	31 Oct 2021

Relationship Between Tables

- A table may be related to other tables
- For example
 - An Employee works in a Department
 - A Manager controls a Project



A Foreign Key

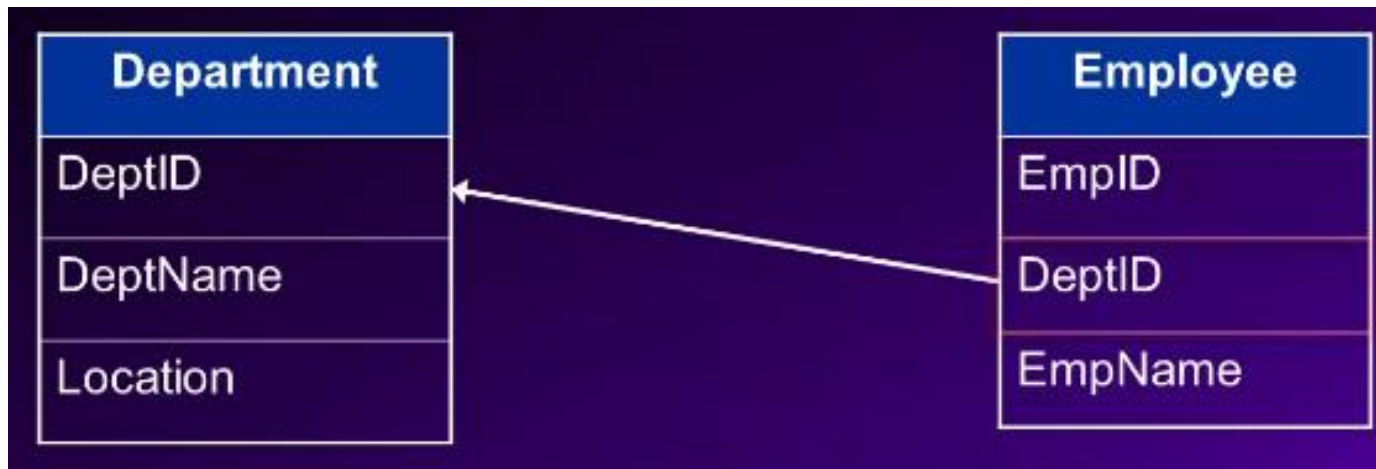
- To establish relationships, we need to implement a foreign key
- A foreign key is a primary key from one table that is placed into another table
- The key is called a foreign in the table that receives the key

Foreign Key Example 1

- A



Foreign Key Example 2



Referential Integrity

- Referential integrity states that every value of a foreign key must match a value of an existing primary key
- A foreign key is a primary key from one table that is placed into another table
- The key is called a foreign in the table that receives the key

Null Values

- A null value means that no data exists
 - You can think of a null values as an empty cell in the table
- This is different from a zero, space character, empty string, or tab character

The Problem of Null Values

- A null is often ambiguous. It could mean...
 - The column value is not appropriate for the specific row
 - The column value has not been decided
 - The column value is unknown
- Each may have entirely different implications

Functional Dependency

- A relationship between attributes in which one attribute (or group of attributes) determines the value of another attribute in same table
- Illustration...
 - The price one delicious Girl Scout cookies can determine the price of a box of 12 cookies

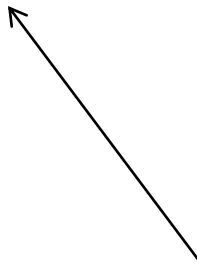
(CookiePrice, Qty) → BoxPrice

Determinants

- The attribute (or attributes) that we use as the starting point (the variable on the left side of the equation) is called a *determinant*

(CookiePrice, Qty)

BoxPrice



Determinant

- By definition...

- A candidate key of relation will functionally determine all other non-key attributes in the row
- Likewise, by definition...

A primary key of a relation will functionally determine all other non – key attributes in the row

Primary Key and Functional Dependency Example

(EmployeeID) $\longrightarrow$ *(EmpLastName, EmpPhone)*

(ProjectID) $\longrightarrow$ *(ProjectName, StartDate)*

Data Normalization

- A process of analyzing a relation to ensure that it is ***well formed***
- Normalization involves decomposing relations with anomalies to produce smaller, well-structured relations
- More specifically, if a relation is normalized (well- formed), rows can be inserted, deleted, or modified without creating anomalies

Normalization Principles

- Relational design principles for normalized relations:
 - To be a well-formed relation, every *determinant* must be a *candidate key*
 - Any relation that is not well – formed should be broken into two or more well – formed relations!
- **TIP:** As a general rule, a well – formed relation will not encompass more than one business concept!

Normalization Example

$(StudentID) \longrightarrow (StudentName, DormName, DormCost)$

However, if...

$(DormName) \longrightarrow (DormCost)$

Then DormCost should be placed into its own relation, resulting in the relations:

$(StudentID) \longrightarrow (StudentName, DormName)$

$(DormName) \longrightarrow (DormCost)$

$(AttorneyID, ClientID) \longrightarrow (ClientName, MeetingDate, Duration)$

However, if...

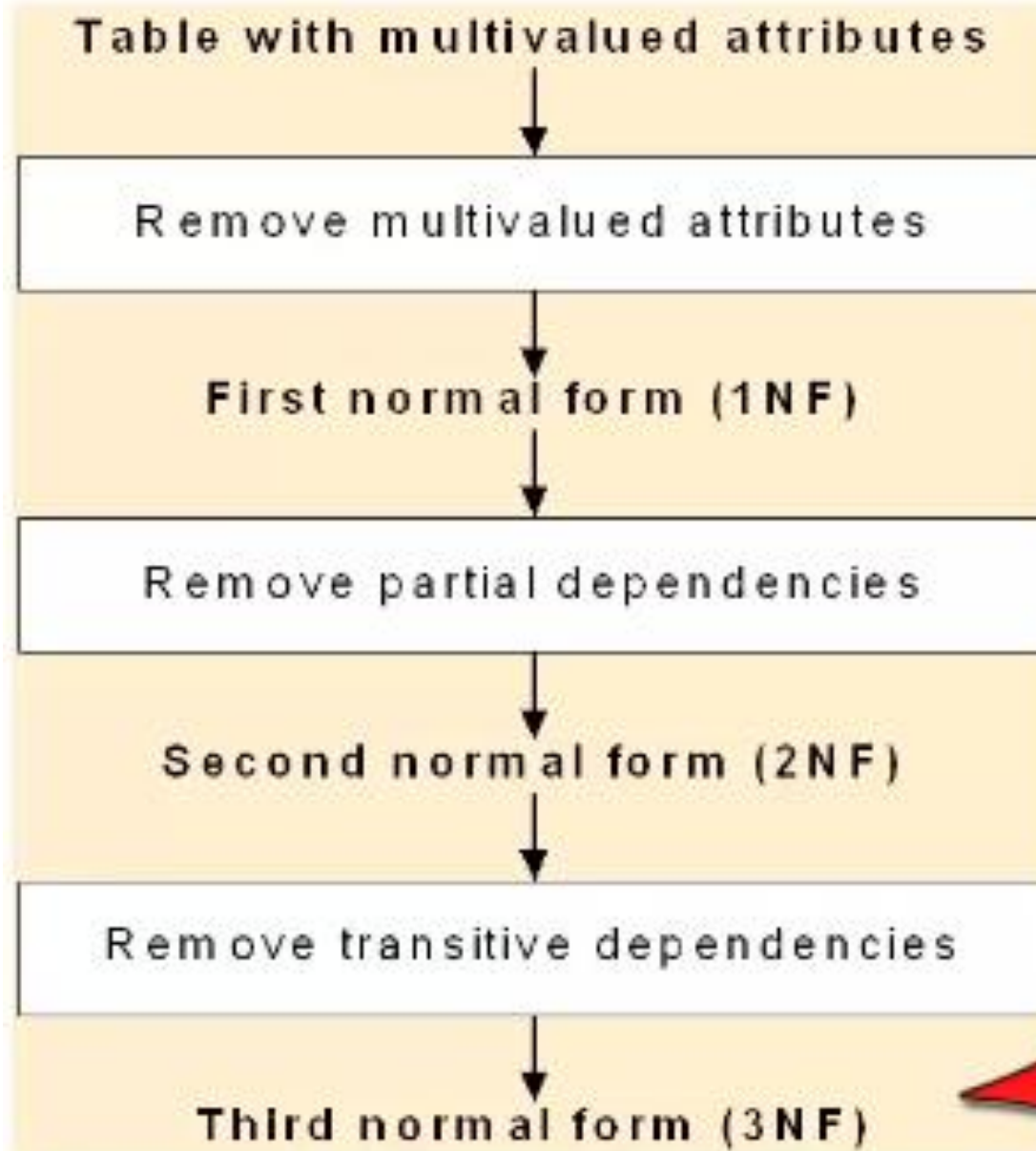
$(ClientID) \longrightarrow (ClientName)$

Then ClientName should be placed into its own relation, resulting in the relations:

$(AttorneyID, ClientID) \longrightarrow (MeetingDate, Duration)$

$(ClientID) \longrightarrow (ClientName)$

Normalization Steps



Goal



First Normal Form

- Table contains no multivalued attributes
- Every attribute value is atomic
- The figure on the next slide is not in First Normal Form (multivalued attributes) *> it is not a relation*
- *All relations are in First Normal Form*

Table with Multivalued Attributes: Not in 1st Normal Form

Order ID	Order Date	Customer ID	Customer Name	Customer Address	Product ID	Product Description	Product Finish	Unit Price	Ordered Quantity
1006	24 Oct 2018	22	Dan's Furniture	Fullerton, CA	7	Tea Table	Walnut	\$450	2
					5	TV Stand	Oak	\$300	6
					4	Porch Swing	Pine	\$800	5
1007	31 Oct 2018	65	Furniture Barn	Ft. Collins, CO	11	Dining Table	Cherry	\$1150	3
					4	Porch Swing	Pine	\$800	7

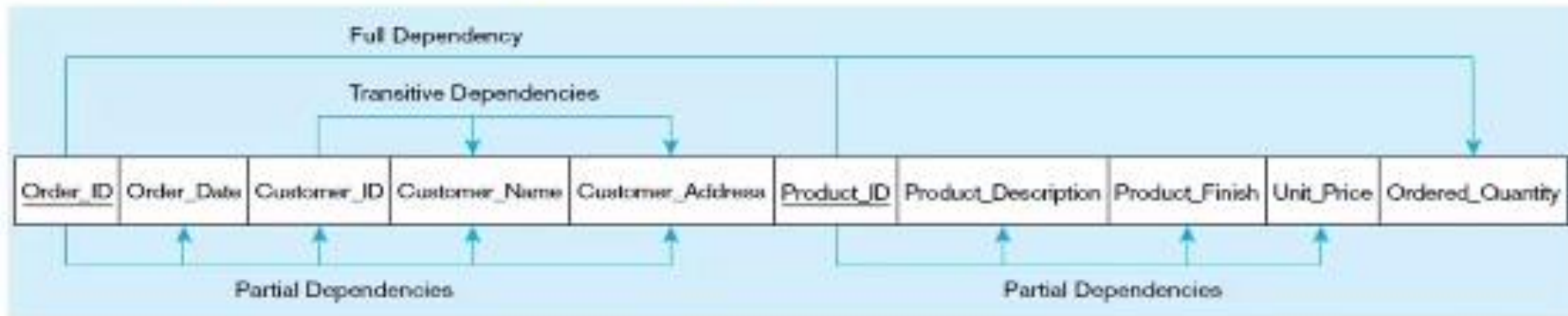
Table with No Multivalued Attributes: In 1st Normal Form

Order ID	Order Date	Customer ID	Customer Name	Customer Address	Product ID	Product Description	Product Finish	Unit Price	Ordered Quantity
1006	24 Oct 2018	22	Dan's Furniture	Fullerton, CA	7	Tea Table	Walnut	\$450	2
1006	24 Oct 2018	22	Dan's Furniture	Fullerton, CA	5	TV Stand	Oak	\$300	6
1006	24 Oct 2018	22	Dan's Furniture	Fullerton, CA	4	Porch Swing	Pine	\$800	5
1007	31 Oct 2018	65	Furniture Barn	Ft. Collins, CO	11	Dining Table	Cherry	\$1150	3
1007	31 Oct 2018	65	Furniture Barn	Ft. Collins, CO	4	Porch Swing	Pine	\$800	7

Second Normal Form

- 1NF PLUS *every non-key attribute is fully functionally dependent on the ENTIRE primary key*
- Every *non-key attribute* must be defined by the entire key, not by only part of the key
- No partial dependencies

Dependency Diagram



Order_ID → Order_Date, Customer_ID, Customer_Name, Customer_Address

Customer_ID → Customer_Name, Customer_Address

Product_ID → Product_Description, Product_Finish, Unit_Price

Order_ID, Product_ID → Order_Quantity

Therefore, NOT in 2nd Normal Form!

Removing Partial Dependencies

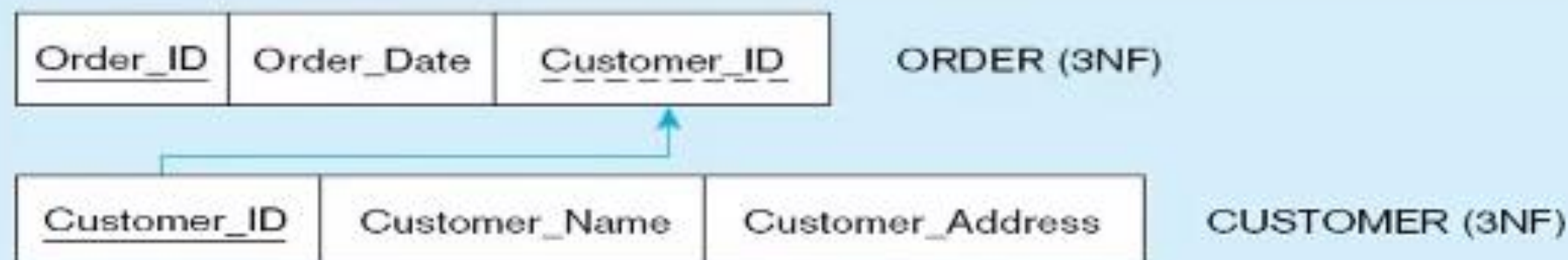


Partial dependencies have been removed, but there are still transitive dependencies in the CUSTOMER_ORDER table...

Third Normal Form

- 2NF PLUS *no transitive dependencies* (functional dependencies on non-primary – key attributes)
- *Such dependencies are called transitive,* because the primary key is a determinant for a third
- **Solution:** Non-key determinant with transitive dependencies goes into a new table; non-key determinant becomes primary key in the old table

Removing Transitive Dependencies



Transitive dependencies have been removed!